

Iteration and Loop Control Structures in C

📌 Definition to Remember

Loops are control structures that repeatedly execute a block of code as long as a specified condition remains true. C supports entry-controlled (for, while) and exit-controlled (do-while) loops.

1. Concept Overview & Fundamentals 🗨️

Iteration and Loop Control Structures in C forms a core cornerstone of computer systems programming, hardware-level software engineering, and competitive problem solving.

In the C execution model:

- Code is compiled directly into native machine instructions without virtual machine overhead.
- Memory is strictly partitioned into **Code/Text Segment**, **Data Segment (Initialized/BSS)**, **Stack**, and **Heap**.
- Direct pointer and register manipulation provides deterministic performance critical for operating systems, embedded hardware, and database kernels.

2. Technical Architecture & Memory Mechanics ⚙️

Memory Layout & Storage Organization

```
+-----+
|                STACK SEGMENT                |
| - Local variables, function call stack frames, parameters |
| - High memory growing downwards towards Heap |
+-----+
|
| v
| ^
|
+-----+
|                HEAP SEGMENT                |
| - Dynamic memory allocations via malloc(), calloc() |
| - Low memory growing upwards towards Stack |
+-----+
|                INITIALIZED DATA SEGMENT        |
| - Global and static variables with non-zero initial values |
+-----+
|                BSS SEGMENT (UNINITIALIZED)        |
| - Global and static variables initialized to zero |
+-----+
|                CODE / TEXT SEGMENT                |
| - Read-only executable binary instructions |
+-----+
```

Core Architecture Breakdown

Component / Concept	Technical Specification	Operational Characteristics	Performance / Complexity
Storage Allocation	Stack vs Heap vs Data Segment	Stack is ultra-fast LIFO; Heap offers dynamic scalability	Stack: O(1) push/pop; Heap: O(1) alloc
Type Checking	Static Typing	Compiler verifies types during compilation	Zero runtime type penalty
Addressing Model	Byte-addressable physical RAM	Pointer stores hexadecimal byte memory address	Direct memory dereference (O(1))
Execution Flow	Procedural / Sequential	Controlled via jumps, branches, loops, and call stacks	Deterministic instruction cycles

3. Core Syntax & Implementation Patterns

Essential Idioms & Header Declarations

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>

// Core standard implementation pattern
int main(void) {
    // Explicit initialization
    int status = 0;

    // Core logic execution
    printf("Executing C Module: Iteration & Loop Control Structure\n");

    return status;
}
```

4. Real-World Applications & System Engineering

- **Operating System Kernels:** Linux, UNIX, and Windows NT kernels utilize C for drivers, process scheduling, and memory paging.
- **Embedded & IoT Devices:** Microcontrollers with constrained RAM (8KB-64KB) require C's predictable footprint.
- **Database Engine Kernels:** High-throughput storage engines (PostgreSQL, SQLite, MySQL InnoDB) rely on low-level buffer pool managers written in C.
- **Game Engine Foundations:** Graphics rasterization, physics engines, and GPU interfaces utilize C/C++ memory arrays for zero-overhead computation.

5. Comprehensive Solved Problems & Code Walkthroughs

Problem 1: ArmstrongNumber

Problem Description & Objective:

Program to check if a number is an Armstrong number

C Code Implementation:

```
//Program to check if a number is an Armstrong number

#include<stdio.h>
int main() {
    /*
     * Logic:
     * - Goal: Program to check if a number is an Armstrong number
     * - Prompts the user for required inputs.
     * - Executes standard control flow, conditions, or loops to compute the result.
     * - Prints the formatted output to the console.
     */

    int num, originalNum, rem, result = 0;

    printf("Enter a number: ");
    scanf("%d", &num);

    originalNum = num;

    while(originalNum != 0) {
        rem = originalNum % 10;
        result = result + (rem * rem * rem);
        originalNum = originalNum / 10;
    }

    if(result == num)
        printf("%d is an Armstrong Number", num);
    else
        printf("%d is not an Armstrong Number", num);

    return 0;
}
```

Problem 2: EvenPrint

Problem Description & Objective:

2. Program using continue to print only even numbers

C Code Implementation:

```
// 2. Program using continue to print only even numbers

#include<stdio.h>

int main() {
    /*
     * Logic:
     * - Goal: 2. Program using continue to print only even numbers
     * - Prompts the user for required inputs.
     * - Executes standard control flow, conditions, or loops to compute the result.
     * - Prints the formatted output to the console.
     */

    int n;

    printf("Enter limit: ");
    scanf("%d", &n);

    printf("Even numbers are:\n");

    for(int i = 1; i <= n; i++) {

        if(i % 2 != 0) {
            continue;
        }

        printf("%d ", i);
    }

    return 0;
}
```

Problem 3: Factorial

Problem Description & Objective:

Write a function that calculates the factorial of a given number.

C Code Implementation:

```
//Write a function that calculates the factorial of a given number.

#include<stdio.h>
int main(){
    /*
     * Logic:
     * - Goal: Write a function that calculates the factorial of a given number.
     * - Prompts the user for required inputs.
     * - Executes standard control flow, conditions, or loops to compute the result.
     * - Prints the formatted output to the console.
     */

    printf("Factorial\n\n");

    int num;
    printf("Enter number: ");
    scanf("%d",&num);

    int fact = 1;
    for(int i = 1;i <= num;i++){
        fact *= i;
    }
    printf("Factorial is %d",fact);

    return 0;
}
```

Problem 4: Fibonacci

Problem Description & Objective:

Program to print Fibonacci series up to a certain number

C Code Implementation:

```

//Program to print Fibonacci series up to a certain number

#include<stdio.h>

int main() {
    /*
     * Logic:
     * - Goal: Program to print Fibonacci series up to a certain number
     * - Prompts the user for required inputs.
     * - Executes standard control flow, conditions, or loops to compute the result.
     * - Prints the formatted output to the console.
     */

    int n, first = 0, second = 1, next;

    printf("Enter limit: ");
    scanf("%d", &n);

    printf("Fibonacci Series: ");

    while(first <= n) {
        printf("%d ", first);

        next = first + second;
        first = second;
        second = next;
    }

    return 0;
}

```

Problem 5: GCD

Problem Description & Objective:

Create a program to find the Greatest Common Divisor (GCD) of two integers.

C Code Implementation:

```

//Create a program to find the Greatest Common Divisor (GCD) of two integers.

#include <stdio.h>

int main() {
    /*
     * Logic:
     * 1. Reads two integers 'first' and 'second' with scanf validation to ensure inputs are numeric.
     * 2. Converts inputs to absolute values ('temp_first', 'temp_second') to support negative
coordinates safely.
     * 3. Implements the iterative Euclidean Algorithm:
     *   - While the divisor (temp_second) is not 0, computes the remainder: temp_first % temp_second.
     *   - Shifts temp_second into temp_first, and the remainder into temp_second.
     *   - This reduces complexity from O(min(a,b)) to logarithmic time O(log(min(a,b))).
     * 4. The final value left in 'temp_first' is the Greatest Common Divisor (GCD).
     */
    int first, second;

    printf("Welcome to GCD Calculator\n");

    printf("Please enter the first number: ");
    if (scanf("%d", &first) != 1) {
        printf("Invalid input.\n");
        return 1;
    }

    printf("Now, enter the second number: ");
    if (scanf("%d", &second) != 1) {
        printf("Invalid input.\n");
        return 1;
    }

    int temp_first = (first < 0) ? -first : first;
    int temp_second = (second < 0) ? -second : second;

    while (temp_second != 0) {
        int temp = temp_second;
        temp_second = temp_first % temp_second;
        temp_first = temp;
    }

    int gcd = temp_first;

    printf("The GCD of %d and %d is %d\n", first, second, gcd);

    return 0;
}

```

Problem 6: InfiniteLoop

Problem Description & Objective:

3. Program using infinite loop and break statement

C Code Implementation:

```
// 3. Program using infinite loop and break statement

#include<stdio.h>

int main() {
    /*
     * Logic:
     * - Goal: 3. Program using infinite loop and break statement
     * - Prompts the user for required inputs.
     * - Executes standard control flow, conditions, or loops to compute the result.
     * - Prints the formatted output to the console.
     */

    int num;

    while(1) {

        printf("Enter a number (-1 to exit): ");
        scanf("%d", &num);

        if(num == -1) {
            break;
        }

        printf("Square = %d\n", num * num);
    }

    printf("Program Ended");

    return 0;
}
```

Problem 7: LCM

Problem Description & Objective:

Create a program to find the Least Common Multiple (LCM) of two numbers.

C Code Implementation:

```
//Create a program to find the Least Common Multiple (LCM) of two numbers.

#include <stdio.h>

int main() {
    /*
     * Logic:
     * - Goal: Create a program to find the Least Common Multiple (LCM) of two numbers.
     * - Prompts the user for required inputs.
     * - Executes standard control flow, conditions, or loops to compute the result.
     * - Prints the formatted output to the console.
     */

    int first, second;

    printf("Welcome to LCM Calculator\n");

    printf("Please enter the first number: ");
    scanf("%d", &first);

    printf("Now, enter the second number: ");
    scanf("%d", &second);

    int min = (first > second) ? first : second;

    int max = first * second;

    for (int i = min; i <= max; i++) {

        if (i % first == 0 && i % second == 0) {

            printf("The LCM of %d and %d is %d\n", first, second, i);

            break;
        }
    }

    return 0;
}
```

Problem 8: Multiplication

Problem Description & Objective:

Develop a program that prints the multiplication table for a given number.

C Code Implementation:

```
//Develop a program that prints the multiplication table for a given number.

#include <stdio.h>
int main() {
    /*
     * Logic:
     * - Goal: Develop a program that prints the multiplication table for a given number.
     * - Prompts the user for required inputs.
     * - Executes standard control flow, conditions, or loops to compute the result.
     * - Prints the formatted output to the console.
     */

    printf("Multiplication Table\n\n");

    int num;

    printf("Enter number: ");
    scanf("%d", &num);

    for (int i = 1; i <= 12; i++){

        printf("%d x %d = %d\n", num, i, (num * i));
    }
    return 0;
}
```

Problem 9: OddSum

Problem Description & Objective:

Create a program to sum all odd numbers from 1 to a specified number N.

C Code Implementation:

```
//Create a program to sum all odd numbers from 1 to a specified number N.

#include <stdio.h>

int main() {
    /*
     * Logic:
     * - Goal: Create a program to sum all odd numbers from 1 to a specified number N.
     * - Prompts the user for required inputs.
     * - Executes standard control flow, conditions, or loops to compute the result.
     * - Prints the formatted output to the console.
     */

    int sum = 0;
    int num;

    printf("Enter value of N: ");
    scanf("%d", &num);

    for (int i = 1; i <= num; i++) {
        if (i % 2 != 0) {
            sum += i;
        }
    }
    printf("Sum of odd numbers = %d\n", sum);

    return 0;
}
```

Problem 10: PalindromeNumber

Problem Description & Objective:

Program to check if a number is palindrome

C Code Implementation:

```

//Program to check if a number is palindrome

#include<stdio.h>
int main() {
    /*
     * Logic:
     * - Goal: Program to check if a number is palindrome
     * - Prompts the user for required inputs.
     * - Executes standard control flow, conditions, or loops to compute the result.
     * - Prints the formatted output to the console.
     */

    int num, originalNum, reverse = 0, rem;

    printf("Enter a number: ");
    scanf("%d", &num);

    originalNum = num;

    while(num != 0) {
        rem = num % 10;
        reverse = reverse * 10 + rem;
        num = num / 10;
    }

    if(originalNum == reverse)
        printf("%d is a Palindrome Number", originalNum);
    else
        printf("%d is not a Palindrome Number", originalNum);

    return 0;
}

```

Problem 11: PatternPrint

Problem Description & Objective:

Implementation of PatternPrint

C Code Implementation:

```

#include<stdio.h>

int main() {
    /*
     * Logic:
     * - Goal: Processes user inputs and calculates results using standard control flows.
     * - Prompts the user for required inputs.
     * - Executes standard control flow, conditions, or loops to compute the result.
     * - Prints the formatted output to the console.
     */

    int rows, i, j;

    printf("Enter number of rows: ");
    scanf("%d", &rows);

    // Right Half Pyramid
    printf("\nRight Half Pyramid:\n");

    for(i = 1; i <= rows; i++) {
        for(j = 1; j <= i; j++) {
            printf("* ");
        }
        printf("\n");
    }

    // Reverse Right Half Pyramid
    printf("\nReverse Right Half Pyramid:\n");

    for(i = rows; i >= 1; i--) {
        for(j = 1; j <= i; j++) {
            printf("* ");
        }
        printf("\n");
    }

    // Left Half Pyramid
    printf("\nLeft Half Pyramid:\n");

    for(i = 1; i <= rows; i++) {
        for(j = 1; j <= rows - i; j++) {
            printf(" ");
        }
        for(j = 1; j <= i; j++) {
            printf("* ");
        }
        printf("\n");
    }

    return 0;
}

```

Problem 12: PrimeNumber

Problem Description & Objective:

Program to check whether a number is prime using while

C Code Implementation:

```
//Program to check whether a number is prime using while

#include<stdio.h>

int main() {
    /*
     * Logic:
     * - Goal: Program to check whether a number is prime using while
     * - Prompts the user for required inputs.
     * - Executes standard control flow, conditions, or loops to compute the result.
     * - Prints the formatted output to the console.
     */

    int num, i = 2, isPrime = 1;

    printf("Enter a number: ");
    scanf("%d", &num);

    if(num <= 1) {
        isPrime = 0;
    }

    while(i < num) {
        if(num % i == 0) {
            isPrime = 0;
            break;
        }
        i++;
    }

    if(isPrime)
        printf("%d is a Prime Number", num);
    else
        printf("%d is not a Prime Number", num);

    return 0;
}
```

Problem 13: ReverseNumber

Problem Description & Objective:

Program to reverse the digits of a number

C Code Implementation:

```

//Program to reverse the digits of a number

#include<stdio.h>

int main() {
    /*
     * Logic:
     * - Goal: Program to reverse the digits of a number
     * - Prompts the user for required inputs.
     * - Executes standard control flow, conditions, or loops to compute the result.
     * - Prints the formatted output to the console.
     */

    int num, reverse = 0, rem;

    printf("Enter a number: ");
    scanf("%d", &num);

    while(num != 0) {
        rem = num % 10;
        reverse = reverse * 10 + rem;
        num = num / 10;
    }

    printf("Reversed Number = %d", reverse);

    return 0;
}

```

Problem 14: SumOfDigit

Problem Description & Objective:

Create a program that computes the sum of the digits of an integer.

C Code Implementation:

```
//Create a program that computes the sum of the digits of an integer.

#include <stdio.h>

int main() {
    /*
     * Logic:
     * - Goal: Create a program that computes the sum of the digits of an integer.
     * - Prompts the user for required inputs.
     * - Executes standard control flow, conditions, or loops to compute the result.
     * - Prints the formatted output to the console.
     */

    printf("Sum of Digits!!!\n\n");

    int num, digit, sum = 0;

    printf("Enter number: ");
    scanf("%d", &num);

    while (num != 0) {

        digit = num % 10;

        sum += digit;

        num = num / 10;
    }

    printf("Sum of digits = %d\n", sum);

    return 0;
}
```

Problem 15: SumPositive

Problem Description & Objective:

1. Program using continue to sum all positive numbers

C Code Implementation:

```
// 1. Program using continue to sum all positive numbers
// Skip negative numbers

#include<stdio.h>

int main() {
    /*
     * Logic:
     * - Goal: 1. Program using continue to sum all positive numbers Skip negative numbers
     * - Prompts the user for required inputs.
     * - Executes standard control flow, conditions, or loops to compute the result.
     * - Prints the formatted output to the console.
     */

    int num, sum = 0;

    printf("Enter 5 numbers:\n");

    for(int i = 1; i <= 5; i++) {

        scanf("%d", &num);

        if(num < 0) {
            continue;
        }

        sum = sum + num;
    }

    printf("Sum of positive numbers = %d", sum);

    return 0;
}
```

Problem 16: even numbers continue

Problem Description & Objective:

Implementation of even numbers continue

C Code Implementation:

```

/*
Create a program using continue to print only even numbers
using continue for odd numbers.
*/

#include <stdio.h>

int main() {
    /*
    * Logic:
    * - Goal: Create a program using continue to print only even numbers using continue for odd numbers.
    * - Prompts the user for required inputs.
    * - Executes standard control flow, conditions, or loops to compute the result.
    * - Prints the formatted output to the console.
    */

    int i;

    printf("Even numbers from 1 to 20:\n");

    for(i = 1; i <= 20; i++) {

        // Skip odd numbers
        if(i % 2 != 0) {
            continue;
        }

        printf("%d ", i);
    }

    return 0;
}

```

Problem 17: multiplication table

Problem Description & Objective:

Implementation of multiplication table

C Code Implementation:

```

/*
Create a program using for loop multiplication table for a number.
*/

#include <stdio.h>

int main() {
    /*
    * Logic:
    * - Goal: Create a program using for loop multiplication table for a number.
    * - Prompts the user for required inputs.
    * - Executes standard control flow, conditions, or loops to compute the result.
    * - Prints the formatted output to the console.
    */

    int num, i;

    printf("Enter a number: ");
    scanf("%d", &num);

    printf("\nMultiplication Table of %d:\n", num);

    for(i = 1; i <= 10; i++) {
        printf("%d x %d = %d\n", num, i, num * i);
    }

    return 0;
}

```

Problem 18: positive number

Problem Description & Objective:

Implementation of positive number

C Code Implementation:

```

/*
Create a program that prompts the user to enter a positive number.
Use a do-while loop to keep asking for the number until
the user enters a valid positive number.
*/

#include <stdio.h>

int main() {
    /*
    * Logic:
    * - Goal: Create a program that prompts the user to enter a positive number. Use a do-while loop to
    keep asking for the number until the user enters a valid positive number.
    * - Prompts the user for required inputs.
    * - Executes standard control flow, conditions, or loops to compute the result.
    * - Prints the formatted output to the console.
    */

    int num;

    do {
        printf("Enter a positive number: ");
        scanf("%d", &num);

        if(num <= 0) {
            printf("Invalid input! Please enter a positive number.\n");
        }

    } while(num <= 0);

    printf("You entered a valid positive number: %d\n", num);

    return 0;
}

```

Problem 19: prime number

Problem Description & Objective:

Implementation of prime number

C Code Implementation:

```

/*
Question 46:
Create a program using for loop to display
if a number is prime or not.
*/

#include <stdio.h>

int main() {
    /*
    * Logic:
    * 1. Confirms the input 'num' is successfully read with scanf.
    * 2. Any integer less than or equal to 1 is automatically not prime (isPrime = 0).
    * 3. Loops through divisors 'i' starting from 2 up to the square root of 'num' ( $i * i \leq \text{num}$ ).
    *    This optimizes the complexity to  $O(\sqrt{n})$  since any composite number must have a factor  $\leq \sqrt{n}$ .
    * 4. If any 'i' divides 'num' perfectly ( $\text{num} \% i == 0$ ), flags the number as non-prime and breaks out
    of the loop.
    * 5. Prints whether the number is a Prime or not.
    */
    int num, i, isPrime = 1;

    printf("Enter a number: ");
    if (scanf("%d", &num) != 1) {
        printf("Invalid input.\n");
        return 1;
    }

    if(num <= 1) {
        isPrime = 0;
    } else {

        for(i = 2; i * i <= num; i++) {
            if(num % i == 0) {
                isPrime = 0;
                break;
            }
        }
    }

    if(isPrime == 1) {
        printf("%d is a Prime Number.\n", num);
    } else {
        printf("%d is NOT a Prime Number.\n", num);
    }

    return 0;
}

```

Problem 20: square break loop

Problem Description & Objective:

Implementation of square break loop

C Code Implementation:

```

/*
Write a program that continuously reads integers from
the user and prints their squares.

Use an infinite loop and a break statement to exit
when a special number (e.g., -1) is entered.
*/

#include <stdio.h>

int main() {
    /*
    * Logic:
    * - Goal: Write a program that continuously reads integers from the user and prints their squares.
    Use an infinite loop and a break statement to exit when a special number (e.g., -1) is entered.
    * - Prompts the user for required inputs.
    * - Executes standard control flow, conditions, or loops to compute the result.
    * - Prints the formatted output to the console.
    */

    int num;

    while(1) {

        printf("Enter a number (-1 to exit): ");
        scanf("%d", &num);

        // Exit condition
        if(num == -1) {
            break;
        }

        printf("Square = %d\n", num * num);
    }

    printf("Program Ended.\n");

    return 0;
}

```

Problem 21: sum positive continue

Problem Description & Objective:

Implementation of sum positive continue

C Code Implementation:

```

/*
Create a program using continue to sum all positive numbers
entered by the user; skip any negative numbers.
*/

#include <stdio.h>

int main() {
    /*
    * Logic:
    * - Goal: Create a program using continue to sum all positive numbers entered by the user; skip any
    negative numbers.
    * - Prompts the user for required inputs.
    * - Executes standard control flow, conditions, or loops to compute the result.
    * - Prints the formatted output to the console.
    */

    int num, sum = 0;

    printf("Enter numbers (0 to stop):\n");

    while(1) {
        scanf("%d", &num);

        // Stop the loop
        if(num == 0) {
            break;
        }

        // Skip negative numbers
        if(num < 0) {
            continue;
        }

        sum = sum + num;
    }

    printf("Sum of positive numbers = %d\n", sum);

    return 0;
}

```

Problem 22: sum until zero

Problem Description & Objective:

Implementation of sum until zero

C Code Implementation:

```

/*
Develop a program that calculates the sum of all numbers
entered by a user until the user enters 0.
The total sum should then be displayed.
*/

#include <stdio.h>

int main() {
    /*
    * Logic:
    * - Goal: Develop a program that calculates the sum of all numbers entered by a user until the user
    enters 0. The total sum should then be displayed.
    * - Prompts the user for required inputs.
    * - Executes standard control flow, conditions, or loops to compute the result.
    * - Prints the formatted output to the console.
    */

    int num, sum = 0;

    do {
        printf("Enter a number (0 to stop): ");
        scanf("%d", &num);

        sum = sum + num;

    } while(num != 0);

    printf("Total Sum = %d\n", sum);

    return 0;
}

```

6. Exam Scoring Points & Memory Revision

★ Must-Write Points

1. for loop: Combines initialization, condition test, and update in a single compact header.
2. while loop: Entry-controlled loop; checks condition before executing body (0 to N iterations).
3. do-while loop: Exit-controlled loop; guarantees at least one execution (1 to N iterations).
4. break: Immediately exits the loop; continue: Skips remaining statements in current iteration.
5. Nested loops: Used for multidimensional arrays, matrix arithmetic, and 2D pattern printing.

⚡ Quick Recall

Init -> Condition Check -> Loop Body -> Step Update -> Break/Continue Control